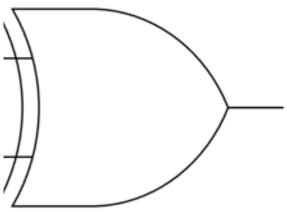
A 1.2.3 Logic Gates



A 1.2.3 Describe the purpose and use of logic gates.

Logic gates are the components that let a computer act on binary signals rather than merely store or move them. Each gate takes one or two binary inputs and produces a single binary output according to one fixed rule, and that rule is the whole of the gate's behaviour. Built from transistors and combined in vast numbers on a single chip, gates are what turn patterns of 0s and 1s into arithmetic, comparison and control. This page describes what logic gates are for, how they are used inside a computer system, the part they play in binary computing, and the seven Boolean operators the syllabus names.

Last Updated: 13 Jun 2026

Key Questions

- What is a logic gate, and what does it do to its inputs?
- What are logic gates used for inside a computer system?
- Why are logic gates fundamental to binary computing?
- What does each of the seven gates (AND, OR, NOT, NAND, NOR, XOR, XNOR) do?

IB Command Term note

The command term here is **describe**: give a detailed account.

For logic gates that means stating clearly what a gate does to its inputs, not just naming it. Saying “this is an AND gate” describes nothing; saying “the output is 1 only when both inputs are 1” does. When a question asks you to describe the purpose or use of logic gates, give their function, controlling binary signals by a logical rule, and a concrete use such as performing arithmetic, rather than reciting a list of gate names.

What a logic gate does

A logic gate is a small electronic circuit that takes binary inputs, usually one or two, and produces a single binary output. The inputs and the output are voltages that the system reads as 0 or 1. The gate applies one fixed logical rule, so the output is decided completely by the current inputs. A gate holds no value and does not change over time: the same inputs always give the same output.

This fixed behaviour is what makes gates useful. A computer represents everything it handles, numbers, characters and instructions, as patterns of 0s and 1s, and to compute is to turn one pattern into another. Gates are the elements that carry out those changes. One gate on its own makes only a trivial decision, but wired together in large numbers gates perform addition, comparison, selection and every other operation a processor carries out.

THINK FIRST

Imagine a circuit that should switch a motor on only when two separate safety switches are both closed. Before reading on, work out what the output should be for each of the four combinations of the two switches. You have just designed an AND gate.

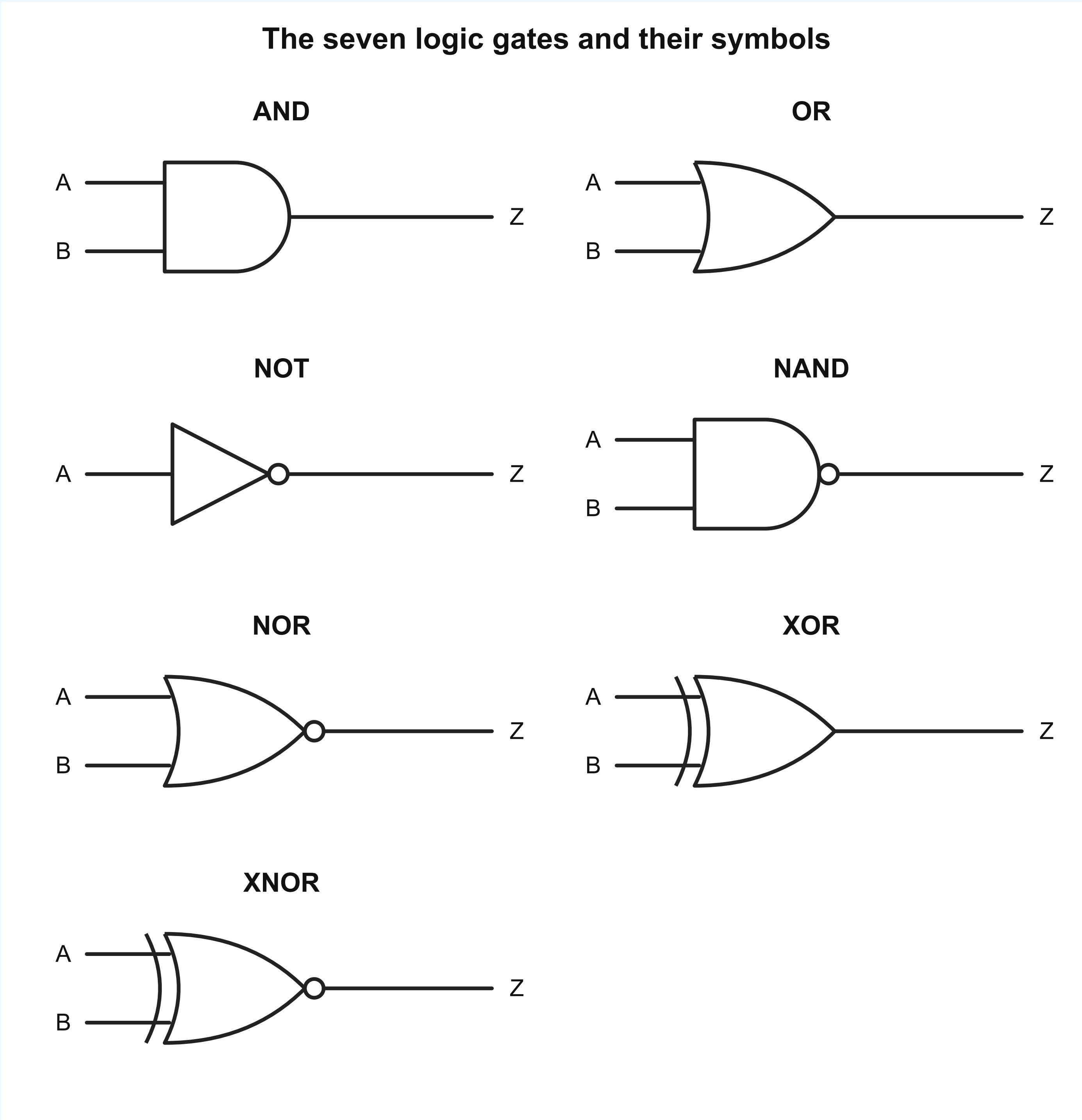
Functions and applications in a computer system

A single gate does little. Gates become useful when many are combined into circuits, each circuit doing one well defined job, and many circuits together forming a processor. The main jobs gates are combined to do include:

- **Arithmetic:** a few gates wired together add two binary digits and produce a carry; repeat and extend that pattern and you have a circuit that adds whole numbers. The rest of arithmetic builds on the same idea.
- **Memory:** with the output of some gates fed back to their inputs, a circuit can hold a value after the input is removed. This is how a single bit of storage is built.
- **Decision and control:** gates implement the conditions behind branching in a program, for example acting only when one signal is 1 and another is 0.
- **Routing:** gates select which of several signals reaches a destination, steering data between the processor, memory and peripherals.

The seven Boolean operators

Each gate corresponds to a Boolean operator, the logical operation it performs. Three operators are the basic ones: AND, OR and NOT. The other four are formed from these. In Boolean notation, AND is written with a dot ($A \cdot B$) or the symbol $\wedge$, OR with a plus ($A + B$) or $\vee$, and NOT with an overbar ($\overline{A}$) or $\neg$. The diagram below shows the standard symbol for each gate; the inputs are labelled A and B and the output Z.



The standard symbols for the seven logic gates. Inputs are labelled A and B, the output Z.

AND

The output is 1 only when both inputs are 1, and 0 otherwise. Use an AND gate to express a condition that needs several things to be true at the same time, such as a door that unlocks only when a valid card is presented and the time is within working hours.

OR

The output is 1 when at least one input is 1, and 0 only when both inputs are 0. An OR gate suits a condition where any one of several things being true is enough, such as an alarm that sounds if either the front or the back sensor is triggered.

NOT

The NOT gate, also called an inverter, is the only gate with a single input. It flips the input: 1 becomes 0 and 0 becomes 1. Use it to express the opposite of a condition.

NAND

NAND is an AND gate followed by a NOT, so its output is the inverse of AND: 0 only when both inputs are 1, and 1 in every other case.

NOR

NOR is an OR gate followed by a NOT, so its output is the inverse of OR: 1 only when both inputs are 0, and 0 in every other case.

XOR

XOR, the exclusive OR, gives an output of 1 when the two inputs differ and 0 when they are the same. It parts company with OR in one case: when both inputs are 1, OR gives 1 but XOR gives 0. XOR appears in binary addition, where adding two 1s gives 0 in that column and a carry.

XNOR

XNOR, the exclusive NOR, is the inverse of XOR. Its output is 1 when the two inputs are the same and 0 when they differ, which makes it a test for equality: the output is 1 exactly when the inputs match.

The table below gives the output of all six two-input gates for every combination of inputs A and B. Reading across a row shows how the gates differ on the same inputs; the bottom row, where both inputs are 1, separates them most clearly.

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

The NOT gate has a single input, so it has its own short table:

A	Z (NOT A)
0	1
1	0

COMMON EXAM MISTAKE

Treating OR and XOR as the same gate. Both give 1 when exactly one input is 1, so they look identical until both inputs are 1. There, OR gives 1 and XOR gives 0. If a description says a gate is “on only when the inputs are different”, that is XOR, not OR. Students routinely confuse the two, and an answer that swaps them will not score.

The role of logic gates in binary computing

Binary computing means representing and processing information as two-state signals, read as 0 and 1. Gates matter because they are the part of the machine that acts on those two states by logical rule. Memory holds binary patterns and buses move them, but neither changes them; gates are what transform one pattern into another. Without gates a computer could store and shift data but could not compute with it.

Because a gate has no memory and no sense of time, its behaviour is captured completely by a truth table, which lists every combination of inputs and the output for each. There is nothing more to know about what a gate does. Truth tables are the focus of the next page, A 1.2.4.

Summary

This table is for revision. For each gate it gives the Boolean notation and the condition under which the output is 1.

Gate	Boolean notation	Output is 1 when
AND	$A \cdot B$ ($A \wedge B$)	both inputs are 1
OR	$A + B$ ($A \vee B$)	at least one input is 1
NOT	$\neg A$ ($\overline{A}$)	the single input is 0
NAND	$\neg(A \cdot B)$	the inputs are not both 1
NOR	$\neg(A + B)$	both inputs are 0
XOR	$A \oplus B$	the two inputs differ
XNOR	$\neg(A \oplus B)$	the two inputs are the same

TRY IT ON PAPER

Answer these on paper, then check against the model answers. These are the written-response questions for this page; the multiple choice quiz follows.

1. Describe what a logic gate does to its inputs. [2 marks]

Model answer: It takes one or more binary inputs, each 0 or 1 [1], and produces a single binary output determined by one fixed logical rule [1].

2. Describe the difference between an OR gate and an XOR gate. [2 marks]

Model answer: An OR gate outputs 1 when at least one input is 1, including when both are 1 [1]. An XOR gate outputs 1 only when the inputs differ, so two inputs of 1 give an output of 0 [1].

3. A computer already has memory to store binary data and buses to move it. Explain why logic gates are still needed. [3 marks]

Model answer: Storing and moving data does not alter it [1]. Computing means transforming one binary pattern into another [1]. Logic gates apply logical rules to binary inputs to produce new outputs, performing the transformation that memory and buses cannot [1].

4. Complete the sentence: a logic gate has no memory, so its behaviour is fully described by a ___________, which lists the output for every combination of inputs. [1 mark]

Model answer: truth table [1].

A 1.2.3 Quiz

1

A logic gate gives an output of 1 only when both of its inputs are 1. Which gate is it?

A. AND

B. OR

C. NAND

D. XOR

AND outputs 1 only when both inputs are 1. NAND is its inverse, giving 0 only when both inputs are 1. OR outputs 1 whenever at least one input is 1, and XOR outputs 1 only when the inputs differ.

2

Which of these gates has only one input?

A. XOR

B. NOT

C. NOR

D. NAND

NOT inverts a single input. XOR, NOR and NAND each take two inputs.

3

For inputs A = 1 and B = 1, which gate gives an output of 0?

A. OR

B. AND

C. XNOR

D. NAND

With both inputs 1, NAND gives 0 because it is the inverse of AND. AND itself gives 1, OR gives 1, and XNOR gives 1 because the inputs are the same. The most tempting wrong answer is AND, chosen by forgetting that NAND inverts the AND result.

4

An OR gate gives an output of 0 only when:

A. both inputs are 1

B. at least one input is 1

C. both inputs are 0

D. the two inputs are different

OR gives 1 if any input is 1, so the only way to get 0 is for both inputs to be 0. “Both inputs are 1” would give an output of 1, and so would option “the two inputs are different”.

5

Which gate gives an output of 1 when its two inputs are the same, acting as an equality test?

A. XOR

B. XNOR

C. NOR

D. AND

XNOR outputs 1 when the inputs match. XOR is the opposite, giving 1 only when they differ. NOR gives 1 only when both inputs are 0, and AND only when both are 1.

6

The NOR gate gives an output of 1 when:

A. both inputs are 1

B. both inputs are 0

C. exactly one input is 1

D. the two inputs differ

NOR is OR followed by NOT. OR gives 0 only when both inputs are 0, so NOR gives 1 only then.